

Self-Introduction

(~20 seconds)

"Good [morning/afternoon], everyone. My name is xxxx, student ID xxxxx, and in this part of the presentation, I'll be walking you through our Wireshark traffic analysis of the SSL/TLS handshake."

Slide 14 — Introduction to Wireshark

(~40 seconds)

"Now that we've covered the theory behind SSL and TLS, I'm going to show you what it actually looks like on the wire, using a tool called Wireshark. Wireshark is a free, open-source network protocol analyzer. In simple terms, it lets us capture every packet travelling across a network connection and inspect it in detail — not just the header, but the full payload too. When Wireshark captures traffic, it decodes each packet layer by layer, following the OSI model — from the Ethernet layer, up through IP and TCP, all the way to the Application layer. For our analysis, we don't want to look at every single packet, so we use a display filter — simply typing `tls` — which isolates only the TLS handshake and application-data records. That's what let us focus specifically on the SSL/TLS handshake we studied earlier."

Slide 15 — Client Hello Analysis

(~55 seconds)

"This first screenshot shows the very first packet of the handshake — the Client Hello. You can see this packet tells the server four important things. First, the TLS Version — here the client is proposing TLS 1.2. Second, the Random value — a 32-byte random number generated by the client, which later gets combined with the server's random value to help derive the session key. Third, the Cipher Suites — this is the list of encryption combinations the client is willing to use. In our capture, the client offered `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`, along with an empty renegotiation info suite for backward compatibility. And finally, the Extensions — additional information like supported elliptic curve groups and signature algorithms, which help the server choose how to respond."

Slide 16 — Server Hello Analysis

(~55 seconds)

"The server replies with its own Server Hello packet, and this is where the negotiation actually gets decided. Notice the server confirms the TLS Version — TLS 1.2, matching the client's proposal. It then picks exactly one Cipher Suite from the client's list — in our capture, it selected `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`. That single choice now fixes how the rest of this session will be encrypted and authenticated. You can also see the Session ID — in this capture it's length zero, meaning no session resumption is being offered here — and the Server

Random, which, combined with the client's random from the previous slide, will feed into the key derivation process."

Slide 17 — Certificate Analysis (~60 seconds)

"Next, the server sends its digital certificate so the client can verify its identity. Looking at the capture, we can identify all the key fields we discussed earlier. The Subject and Issuer here are both listed as 'Test Certificate RSA' — which tells us this is actually a self-signed test certificate, since the issuer and subject are the same entity. In a real production certificate, the issuer would instead be a trusted Certificate Authority. We can also see the Validity period — a ten-year window from September 2013 to September 2023 — and the Public Key, an RSA key with a 65537 exponent, bound to this certificate. Finally, the Signature Algorithm shows sha1WithRSAEncryption — meaning SHA-1 was used to sign this particular certificate."

Slide 18 — Server & Client Key Exchange (~65 seconds)

"Now we move into the key exchange itself, and this is really the heart of the handshake. In the top screenshot, the Server Key Exchange message uses Elliptic Curve Diffie Hellman — you can see the named curve is secp256r1 — and it includes the server's public key parameters, along with a signature proving these parameters genuinely came from the server holding the certificate's private key. Below that, the Client Key Exchange message shows the client sending back its own EC Diffie-Hellman public key. Here's the important part: each side now takes the other's public value and combines it with their own private value to independently compute the same shared secret — without that secret ever being transmitted across the network. That shared secret is then used, together with both random values from earlier, to derive the actual session key that will encrypt everything from this point forward."

Slide 19 — Change Cipher Spec & Finished (~60 seconds)

"This brings us to the final step of the handshake. Both screenshots show a Change Cipher Spec message — one from the client, one from the server. This is essentially a short signal that says: 'from this point onward, everything I send will be encrypted using the session key we just agreed on.' Immediately after, each side sends a Finished message. Notice that Wireshark labels this as an 'Encrypted Handshake Message' — we can't read its contents, and that's actually the whole point. It's the first message protected by the new session key, and it lets each side confirm the handshake completed successfully and wasn't tampered with along the way. Once both Finished messages are exchanged, the handshake is complete, and the connection moves into normal encrypted application data — which is exactly what the next part of our presentation will cover." (Hand over to the next presenter)